identity HTTP/TLS/JSON

Interface Design Description

Service ID: *"identity"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `identity` service of the Authentication system.

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**
2.1 Interface registration . . . 3

3 Service Operations **4**
3.1 operation identity-login . . . 4
3.2 operation identity-logout . . . 4
3.3 operation identity-change-credentials . . . 5
3.4 operation identity-verify . . . 5

4 Data Models **7**
4.1 struct ErrorMessage . . . 7
4.2 struct IdentityChangeRequest . . . 7
4.3 struct IdentityLoginResponse . . . 7
4.4 struct IdentityRequest . . . 7
4.5 struct IdentityVerifyResponse . . . 7
4.6 Primitives . . . 8

5 Go and Rust implementation variants **9**
5.1 Go implementation . . . 9
5.2 Rust implementation . . . 9

6 References **9**

7 Revision History **10**
7.1 Amendments . . . 10
7.2 Quality Assurance . . . 10

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the `identity` service interface of the Authentication system, which provides authentication of systems: a system logs in with its credentials and obtains an identity token, which it presents to other systems; providers verify such tokens.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `identity` service interface

This document provides the Interface Design Description IDD to the *identity – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `identity` service interface in detail. Both in terms of its operations (Section 3) and its information model (Section 4).

2.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https` (`generic_http` when TLS is disabled), access port 8444 and base path `/authentication/identity`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`identity-login`	POST	`/login`
`identity-logout`	POST	`/logout`
`identity-change-credentials`	POST	`/change`
`identity-verify`	GET	`/verify/{token}`

3 Service Operations

The interfaces of the `identity` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `identity` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json`. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

3.1 POST /authentication/identity/login

Operation: identity-login
Input: IdentityRequest
Output: IdentityLoginResponse

Checks the credentials and opens a session. Returns the identity token and its expiry (`identity.token.duration`, 300 s in the default configuration). A previous token of the system becomes invalid.

A successful call returns `200 OK` with the output in the body.

```
POST /authentication/identity/login HTTP/1.1
Host: authentication:8444
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "systemName": "TemperatureSensor",
  "credentials": {
    "password": "<password>"
  }
}
```

Listing 1: An identity-login invocation.

```
{
  "token": "<token>",
  "expirationTime": "2026-10-22T12:00:00Z"
}
```

Listing 2: An identity-login response.

3.2 POST /authentication/identity/logout

Operation: identity-logout
Input: IdentityRequest

Checks the credentials and closes the session of the system.

A successful call returns `200 OK`.

```
POST /authentication/identity/logout HTTP/1.1
Host: authentication:8444
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json
```

```

{
  "systemName": "TemperatureSensor",
  "credentials": {
    "password": "<password>"
  }
}
```

Listing 3: An identity-logout invocation.

3.3 POST /authentication/identity/change

Operation: identity-change-credentials
Input: IdentityChangeRequest

Checks the current credentials and replaces them with the new ones.
A successful call returns `200 OK`.

```
POST /authentication/identity/change HTTP/1.1
Host: authentication:8444
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "systemName": "TemperatureSensor",
  "credentials": {
    "password": "<password>"
  },
  "newCredentials": {
    "password": "<password>"
  }
}
```

Listing 4: An identity-change-credentials invocation.

3.4 GET /authentication/identity/verify/*{token}*

Operation: identity-verify
Input: String
Output: IdentityVerifyResponse

Checks an identity token. Returns whether it is valid and, if so, the system, its sysop flag, login and expiry time. The caller must itself present a valid identity token (only `login`, `logout` and `change` are token-free).
Path parameter: *token*.
A successful call returns `200 OK` with the output in the body.

```
GET /authentication/identity/verify/<token> HTTP/1.1
Host: authentication:8444
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An identity-verify invocation.

```
{
  "verified": true,
  "systemName": "TemperatureSensor",
  "sysop": false,
  "loginTime": "2026-10-22T12:00:00Z",
  "expirationTime": "2026-10-22T12:00:00Z"
}
```

Listing 6: An identity-verify response.

4 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 4.6, which are used to represent things like hashes, identifiers and texts.

4.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

4.2 struct IdentityChangeRequest

Changes the credentials of the requesting system.

Object Field	Value Type	Description
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"credentials"	Object<String>	Credentials of the authentication method, for `PASSWORD` the key `password`.
"newCredentials"	Object<String>	The new credentials.

4.3 struct IdentityLoginResponse

The identity token issued at login.

Object Field	Value Type	Description
"token"	String	The identity token, used as `Bearer IDENTITY-TOKEN//<token>`.
"expirationTime"	DateTime	UTC time when the identity token expires.

4.4 struct IdentityRequest

System name and credentials.

Object Field	Value Type	Description
"systemName"	String	Name of the system, PascalCase (e.g. `TemperatureSensor`).
"credentials"	Object<String>	Credentials of the authentication method, for `PASSWORD` the key `password`.

4.5 struct IdentityVerifyResponse

Result of verifying an identity token.

Object Field	Value Type	Description
"verified"	Boolean	True when the token is valid.
"systemName"	String	System the token belongs to.
"sysop"	Boolean	Whether that system is a system operator.
"loginTime"	DateTime	UTC time of the login.
"expirationTime"	DateTime	UTC time when the identity token expires.

4.6 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	JSON array; elements may be constrained to the stated model or primitive type.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

5 Go and Rust implementation variants

5.1 Go implementation

Primary system folder: `Eclipse-Arrowhead-go/ah5-core-go/authentication`.

The system owns its Go module, launcher and build configuration. Use the per-system `Makefile` and `go.mod`; the containing Go tree is the build source, not a historical Arrowhead copy. Ordinary native roles use independent identities and JSON snapshot state rather than the Java database. Do not assume Java SQL migration, clustering or identical transaction/retry semantics. The registry has its own configuration and optional native AMQP monitor; other roles must be checked against their own registrations.

Listening port. Built-in default 127.0.0.1:8444 (Arrowhead default). The shipped config.json sets "listen": "0.0.0.0:8444". Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. Native design/route evidence: `Eclipse-Arrowhead-go/ah5-core-go/authentication/documentation`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

5.2 Rust implementation

Primary system folder: `Eclipse-Arrowhead-rust/ah5-core-rust/authentication`.

The per-system entry point is built by the `arrowhead-core-rust` crate. Build from the primary Rust workspace with the applicable package/binary selection. Documentation links can resolve into the shared core/support catalogue. Independent process identity and state remain required.

The ordinary native profile uses Tokio/Axum and rustls with peer-identity checks and independent locked JSON state. Network effects are not distributed database transactions. The precise trust, CRL reload and persistence guarantees are those of the native profile, not the Java implementation.

Native design/route evidence: `Eclipse-Arrowhead-rust/ah5-core-rust/documentation/Authentication`.

For this Java interface, select only a native transport and operation explicitly implemented and advertised by that port. A matching method/path does not establish DTO, error, authorization, retry or persistence equivalence. Java MQTT/AMQP bindings are not automatically provided by a Go or Rust role. Differential contract and security checks remain necessary before substitution.

6 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.3.0, 2026, authentication-v5.3.0-identity-SD.pdf.

7 Revision History

7.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

7.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	